Cloud Lab - VPC in AWS

Wednesday, 24 June 2026 7:39 PM

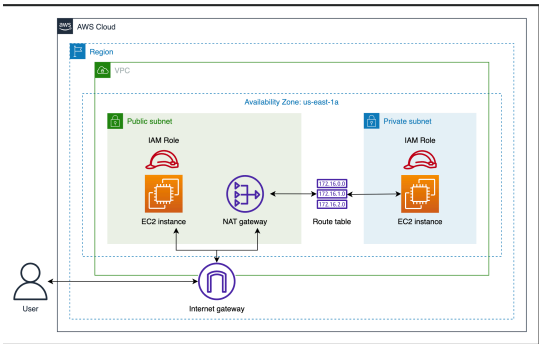
Getting Started with Virtual Private Cloud (VPC) in AWS

Amazon Virtual Private Cloud (Amazon VPC) is a virtual network defined by its users, wherein they can launch AWS resources such as an EC2 instance. It allows users to define network space and configurations to limit access to the AWS resources within a network. Users can also use Amazon's scalable infrastructure and advanced security features to create and secure their virtual networks.

In this Cloud Lab, you'll learn how to create a Virtual Private Cloud (VPC) and secure it using public and private subnets. You'll also learn how to manage routing across subnets within a VPC. Lastly, you'll configure internet access for EC2 instances in your public and private subnets.

By the end of this Cloud Lab, you'll be well-equipped to create and manage virtual networks on AWS. You'll also gain a firm grip on network security and traffic routing within a network.

The following is the high-level architecture diagram of the infrastructure you'll create in this Cloud Lab:



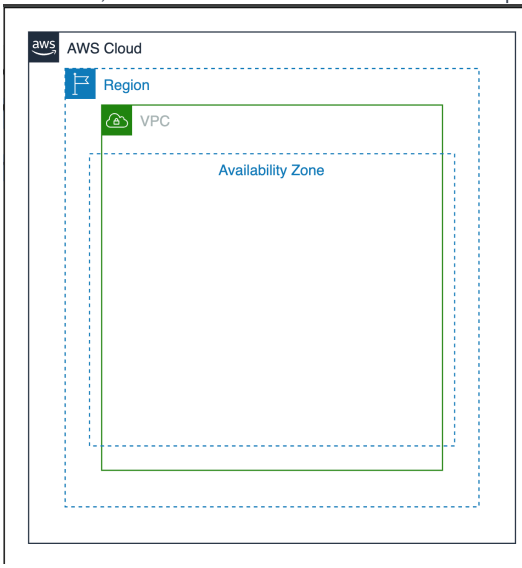
Getting Started

Amazon VPC allows us to create virtual networks over the cloud to isolate and secure the AWS resources. Within a VPC, we can launch resources like EC2 instances. In addition to this, we can configure public and private subnets, routing tables, and internet access to these resources.

We'll start by creating a VPC in this Cloud Lab and attach an internet gateway to it. Next, we'll set up a public subnet within the VPC and update the route tables and subnet associations to route the traffic of resources in the public subnet to an internet gateway. Then, we'll launch an EC2 instance in the public subnet and verify the communication of resources within the public subnet with the internet. Next, we'll launch a private subnet in the VPC. We'll create a NAT gateway in the public subnet. We'll then create a route table for our private subnet to route the traffic from the private subnet to internet gateway via the NAT gateway. Finally, we'll launch an EC2 instance in the private subnet and verify its connection with the internet.

Create a VPC

In this task, we'll create a VPC. The illustration below depicts the provisioned infrastructure of this task.



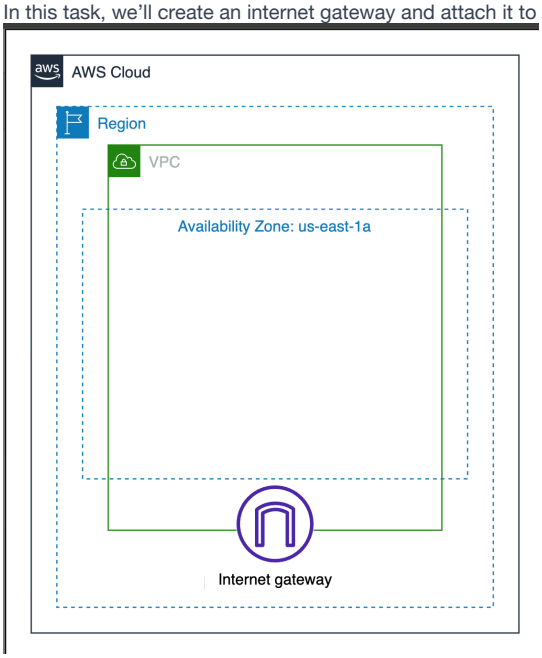
Let's create an instance by following the given steps:

- Head over to the AWS Management Console.
- Search for “VPC” and select “VPC” from the search results. This takes us to the VPC dashboard.
- Click “Your VPCs” under the “Virtual private cloud” heading in the left menu bar.
- Click the “Create VPC” button.
- Configure the following settings under the “VPC settings” section:
    - For “Resources to create,” select “VPC only,” since we’ll only create a VPC in this step. We’ll create other networking resources later.
    - For “Name tag - optional,” enter `mylab-vpc`.
    - For “IPv4 CIDR block,” select “IPv4 CIDR manual input” and enter `10.0.0.0/24` for “IPv4 CIDR.”
    - For “IPv6 CIDR block,” select “No IPv6 CIDR block” from the list.
    - For “Tenancy,” select “Default” from the drop-down list.
- Leave the “Tags” section as it is and click the “Create VPC” button at the end of the page.

We’ll see a success message indicating that our VPC has been successfully created.

Create an Internet Gateway and Attach It to the VPC

The **Internet Gateway** is the component of the VPC that allows the resources within the VPC to connect to the internet. The internet gateway allows both IPv4 and IPv6 traffic to pass through the VPC.
In this task, we’ll create an internet gateway and attach it to our VPC. The provisioned infrastructure is shown below:



Follow the given steps to create an internet gateway:

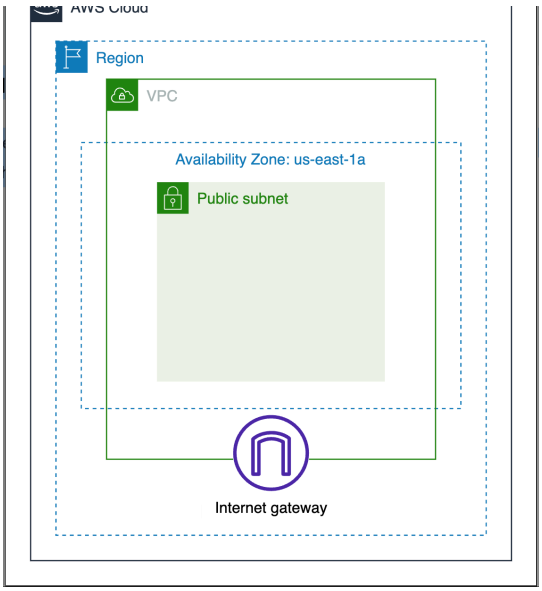
- On the VPC dashboard, click “Internet gateways” under the “Virtual private cloud” heading in the left menu bar. This will redirect us to the internet gateways page, where we’ll be able to see a list of all internet gateways.
- Click the “Create internet gateway” button.
- For “Name tag” in the “Internet gateway settings” section, enter `mylab-internet-gateway`.
- Leave the “Tags - optional” section as it is and click the “Create internet gateway” button at the end of the page.

We’ll now see a success message indicating that our internet gateway has been successfully created. The next step is to attach it to the VPC that we created earlier. Follow the steps given below to attach internet gateway to the VPC:

- Click the “Actions” button, and from the drop-down list, click “Attach to VPC.”
- Select `mylab-vpc` from the drop-down list for “Available VPCs.”
- Click the “Attach internet gateway” button.

Create a Public Subnet

In this task, we’ll create a public subnet. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



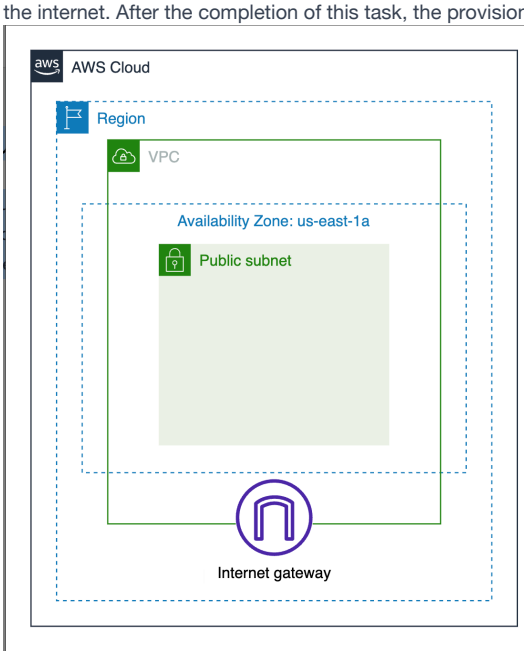
Follow the given steps to create a public subnet:

- Click “Subnets” under the “Virtual private cloud” heading in the left menu bar. This will redirect us to the Subnets page, where we’ll be able to see a list of all subnets.
- Click the “Create subnet” button.
- For “VPC ID” in the “VPC” section, select the VPC “mylab-vpc” we created earlier from the drop-down list.
- Configure the following subnet settings:
  - For “Subnet name,” enter `mylab-public-subnet`.
  - For “Availability Zone,” select “US East (N. Virginia) / us-east-1a” from the drop-down list.
  - For “IPv4 subnet CIDR block,” enter `10.0.0.0/25`.
- Click the “Create subnet” button at the end of the page.

We’ve successfully created a public subnet in our VPC. We’ll now update the associated route table and subnet associations.

Update Route Table and Subnet Associations

Let’s update the route table of our public subnet to send all subnet traffic to the internet gateway. This will allow the instances in our subnet to access the internet. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Update route table

Let’s update the route table of our public subnet by following the given steps:

- Click “Route tables” under the “Virtual private cloud” heading in the left menu bar. This will redirect us to the Route tables page.

- Search for the route table associated with our public subnet by typing `mylab-vpc` in the search bar.

Note: Click the refresh icon if the route table does not appear.

- Click the pencil icon under the “Name” section, provide `mylab-route-table` in the “Edit Name” pop-up and click the “Save” button.
- Click the ID of the route table from the search results.
- Under the “Routes” section, click the “Edit routes” button.

Here, we’ll see a single route entry that will have a “Destination” value of `10.0.0.0/24` and a “Target” value of “local.” This is the default entry that allows instances within our subnet to communicate with other instances in our VPC using private IP addresses.

We’ll now add a second route in this table that will send all other subnet traffic to the internet gateway, which we created earlier. This will allow the instances in our subnet to access the internet via the internet gateway.

- Click the “Add route” button.
- For “Destination,” enter `0.0.0.0/0`.
- For “Target,” select “Internet Gateway” from the drop-down list and then select `mylab-internet-gateway`, which is the name tag of our internet gateway.
- Click the “Save changes” button.

You’ll now see a success message indicating that the routes have been successfully updated.

Subnet associations

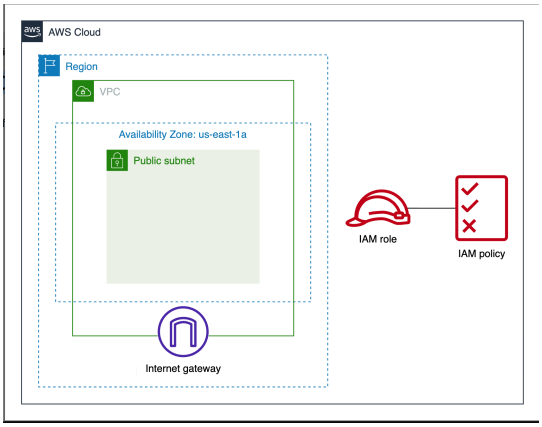
Next, we must explicitly associate this route table with our public subnet.

- Click the “Subnet associations” tab.
- Click the “Edit subnet associations” button under the “Explicit subnet associations” section.
- Select the `mylab-public-subnet` from the list of available subnets.
- Click the “Save associations” button.

With that, we’ve successfully updated the subnet associations.

Next, we’ll create and launch an EC2 instance in the public subnet. In order to do so, we’ll need to create an IAM role to allow AWS SSM to perform actions on our EC2 instance. This will enable us to run commands on our EC2 instance to verify whether or not it has internet access.

Create an IAM Role



Follow the given steps to create an IAM role:

- Search for “IAM” and select “IAM” from the search results. This takes us to the IAM dashboard.

Note: Don’t worry about any errors/warnings you may see on this page.

- Click “Roles” under the “Access management” heading in the left menu bar.
- Click the “Create role” button.
  - For “Trusted entity type,” select “AWS service.”
  - For “Use case,” select “EC2.”
  - Click the “Next” button.
- On the “Add permissions” page, search for “ssmmanagedinstancecore” under the “Permissions policies” section.
- Select the “AmazonSSMManagedInstanceCore” policy from the search results and click the “Next” button.
- Name the role `mylab-role`.
- Scroll to the end of the page and click the “Create role” button.

You’ll see a confirmation message upon the successful creation of the IAM role.

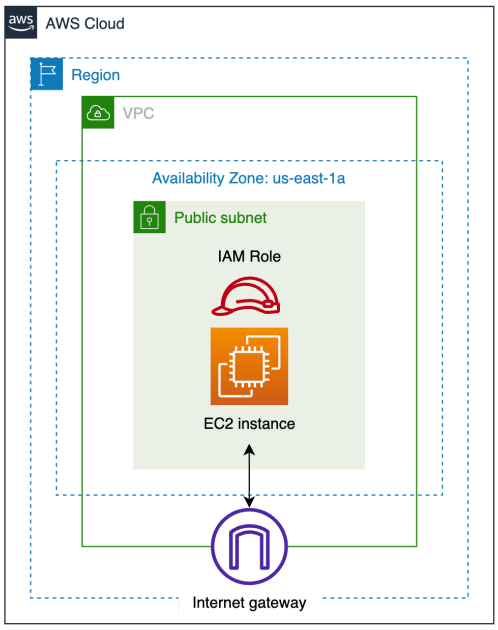
Next, let’s create an EC2 instance in the public subnet.

Next, let's create an EC2 instance in the public subnet.

Create an EC2 Instance

The **Elastic Compute Cloud (EC2)** instance is a scalable computing resource in the cloud provided by AWS. It can be considered as a virtual server that can be configured according to user needs.

In this task, we'll launch an EC2 instance in the public subnet. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Follow the given steps to create and launch an EC2 instance:

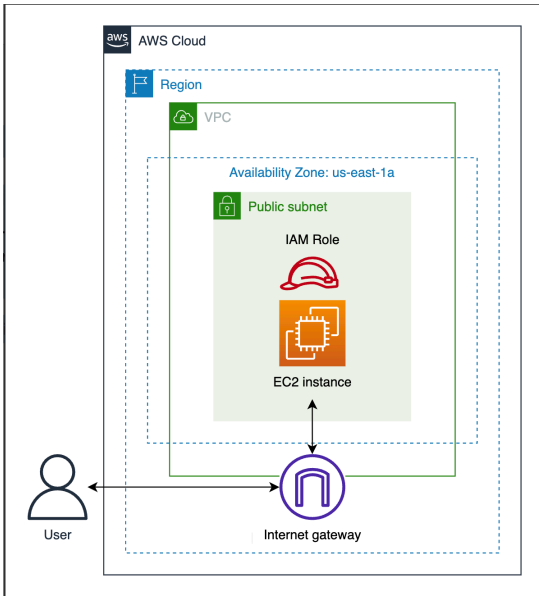
- Search for “EC2” and select “EC2” from the search results. This takes us to the EC2 dashboard.
- Click “Instances” under the “Instances” heading in the left menu bar.
- Click the “Launch instances” button.
- Name the instance `mylab-ec2-01`.
- For “Application and OS Images (Amazon Machine Image),” select “Amazon Linux 2023 kernel-6.1 AMI.”
  - For “Architecture,” select “64-bit (x86).”
- For “Instance type,” select `t3.micro`
- For “Key pair name - required,” under the “Key pair (login)” section, select “Proceed without a key pair (Not recommended)” from the drop-down list.
- Under the “Network settings” section, click the “Edit” button and do the following:
  - For “VPC - required,” select `mylab-vpc` from the drop-down list.
  - For “Subnet,” select `mylab-public-subnet` from the drop-down list.
  - For “Auto-assign public IP,” select “Enable” from the drop-down list since we want the EC2 instance to automatically receive a public IP address when it is launched to enable it to communicate with the internet.
  - For “Firewall (security groups)”, select “Create security group.”
  - Click the “Remove” button under “Inbound security groups rules” to remove the existing rules.
- In the “Configure storage” section, make sure that “gp3,” with 8 GiB storage, is selected as the storage type.
- Click the “Advanced details” section and select `mylab-role` from the drop-down list for “IAM instance profile.”
- Leave the remaining settings as they are and click the “Launch instance” button at the end of the page.

Now that we’ve successfully launched an EC2 instance, we can go ahead and execute a command to verify if the instance has access to the internet or not.

Execute a Command to Test the Internet

AWS Systems Manager (SSM) allows us to manage AWS resources, such as EC2 instances, by automating various operational tasks. It allows us to manage EC2 instances from a centralized location without the need for any additional software or agents. SSM enables us to execute commands remotely on our EC2 instance to perform tasks, such as software installations, and updates, among others.

After the completion of this task, this is what our architecture diagram looks like:



We'll now use the AWS SSM to execute a command on the EC2 instance in the public subnet. We'll try to download the security patches and updates to the EC2 instance, thereby verifying if it has access to the internet or not.

- Search for "Systems Manager" and select "Systems Manager" from the search results. This takes us to the Systems Manager dashboard.
- Click "Run Command" under the "Node Tools" heading in the left menu bar.
- Click the "Run a command" button. This redirects us to the "Run a command" page.
- Under the "Command document" section, search for "runshellscript" using the search bar.
  - Select "AWS-RunShellScript" from the search results.
- Under the "Command parameters" section, type the following command to download the security patches and updates to our EC2 instance.

```
sudo yum update -y
```

- Under the "Target selection" section, do the following:
  - For "Target selection," select "Choose instances manually."
  - For "Instances," select `mylab-ec2-01` from the list.

Note: Sometime the instance is not visible in the "Target selection" section. You'll have to wait for around 10 minutes for it to appear.

- Under the "Output options" section, uncheck the option "Enable an S3 bucket," since we do not want to write the command output to an S3 bucket.
- Click the "Run" button at the end of the page.

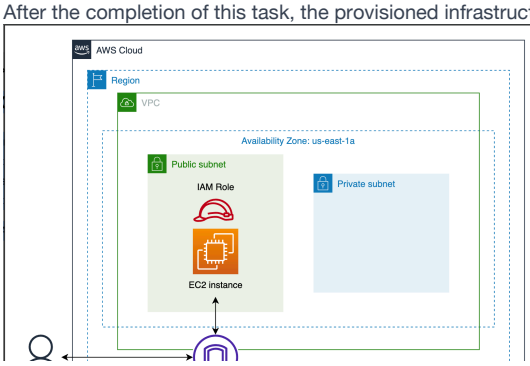
You'll see a success message indicating that our command has been successfully sent. Moreover, under the "Targets and outputs" section, you'll see the "Status" and "Detailed Status" showing "In Progress."

Wait for a little while and press the button with the refresh icon at the top. You'll now see the "Status" and "Detailed Status" showing "Success." This means that we have successfully executed the command on our EC2 instance, which indicates that our EC2 instance successfully accessed the internet to download the security patches and updates.

Create a Private Subnet

In this task, we'll create a private subnet with an EC2 instance and a NAT gateway in the public subnet. We'll update the route table to direct all traffic from the private subnet to the NAT gateway in the public subnet. The public subnet's traffic will then be sent to the internet gateway to allow internet access. We'll then launch an EC2 instance in the private subnet to see if it has access to the internet.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Let’s create a second subnet. This will be a private subnet.

- Search for “VPC” and select “VPC” from the search results to head over to the VPC dashboard.
- Click “Subnets” under the “Virtual private cloud” heading in the left menu bar to view the subnets page.
- Click the “Create subnet” button.
- For “VPC ID” in the “VPC” section, select `mylab-vpc` from the drop-down list.
- Configure the following subnet settings:
  - For “Subnet name,” enter `mylav-private-subnet`.
  - For “Availability Zone,” select “US East (N. Virginia) / us-east-1a” from the drop-down list.
  - For “IPv4 subnet CIDR block,” enter `10.0.0.128/25`.
- Click the “Create subnet” button at the end of the page.

We’ve successfully created a private subnet. We’ll now create a NAT gateway in the public subnet.

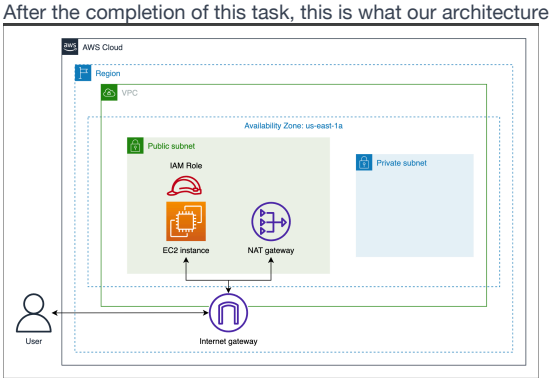
Create a NAT Gateway

A Network Address Translation (NAT) gateway in AWS is a managed service that allows instances in private subnets to access the internet while preventing the internet from initiating connections to those instances, acting as a bridge between private and public networks. There are two types of NAT gateways:

- **Public NAT gateway:** Allows instances in a private subnet to access the internet while blocking access from external traffic.
- **Private NAT gateway:** Allows instances in a private subnet to access AWS resources in other VPCs. It can also allow access to on-premises networks through a transit gateway or a virtual private gateway.

In this task, we’ll create a public NAT gateway in the public subnet to allow the EC2 instance in our private subnet to access the internet through the NAT gateway. Our EC2 instance will be connected to this NAT gateway, which will be connected to the internet gateway to allow internet access to our EC2 instance.

After the completion of this task, this is what our architecture diagram looks like:



Follow the given steps to create a NAT gateway:

- Click “NAT gateways” under the “Virtual private cloud” heading in the left menu bar. This will redirect us to the NAT gateways page, where we’ll be able to see a list of all NAT gateways.
- Click the “Create NAT gateway” button.
- Configure the following settings for the NAT gateway under the “NAT gateways settings” section:
  - For “Name - optional,” enter `mylab-nat-gateway`.
  - For “Availability mode,” select “Zonal” option.
  - For “Subnet,” select `mylab-public-subnet` from the drop-down list.
  - For “Connectivity type,” select “Public” as we want to create a public NAT gateway.
  - For “Elastic IP allocation ID,” click the “Allocate Elastic IP” button to automatically assign an Elastic IP to our public NAT gateway.
- Leave the “Tags” section as it is and click the “Create NAT gateway” button at the end of the page.

We’ll now see a success message indicating that our NAT gateway has been successfully created.

Note: The initial status of the NAT gateway is “Pending.” We can only use it once it changes to “Available.”

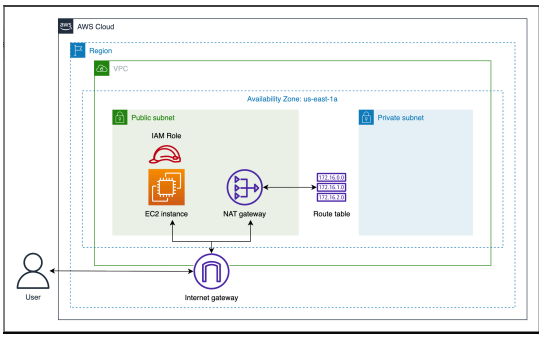
Next, we’ll create a new route table for the private subnet and update the route table to redirect all traffic to the NAT gateway.

Create a Route Table and Update Routes and Subnet Associations

The two subnets we have created so far are both associated with the same route table. You can verify this by checking the value of the “Route table” column for the two subnets on the “Subnets” page.

We will now create a second custom route table and associate it with our private subnet. The original route table will remain associated with the public subnet. We need to create separate route tables for the two subnets because the private subnet’s traffic will be sent to the NAT gateway in the public subnet. The public subnet’s traffic will then be sent to the internet gateway to allow internet access.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



We’ll now create a new custom route table and associate it with the private subnet. We’ll also update this route table to send all subnet traffic to the NAT gateway in the public subnet.

- Click the “Route tables” under the “Virtual private cloud” heading in the left menu bar. This will redirect us to the route tables page.
- Click the “Create route table” button.
- Name the table `mylab-custom-route-table`.
- For “VPC,” select `mylab-vpc` from the drop-down list.
- Click the “Create route table” button at the end of the page.

You’ll see a success message indicating the successful creation of our custom route table. Next, we must update the routes and associate this route table with our private subnet.

Click the “Edit routes” button under the “Routes” section on our custom route page. Here, you’ll see a single route entry that will have a “Destination” value of “10.0.0.0/24” and a “Target” value of “local.” This is the default entry that allows communication between instances in our subnet and those in our VPC using private IP addresses.

We’ll now add a second route in this table that will send all other subnet traffic to the NAT gateway, which we created earlier.

- Click the “Add route” button.
- For “Destination,” enter `0.0.0.0/0`.
- For “Target,” select “NAT Gateway” from the drop-down list and then select `mylab-nat-gateway`, which is the name of our NAT gateway.
- Click the “Save changes” button.

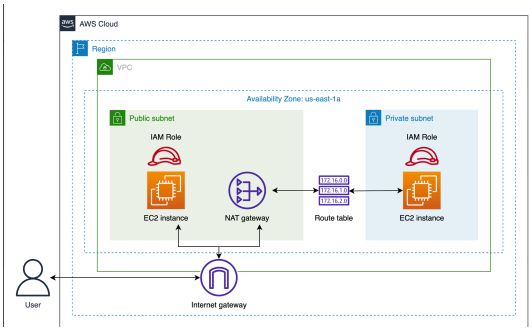
We’ve successfully updated the route table. Now, let’s explicitly associate this route table with our private subnet.

- Click the “Subnet associations” tab.
- Click the “Edit subnet associations” button under the “Explicit subnet associations” section.
- Select the `mylab-private-subnet` from the list of available subnets.
- Click the “Save associations” button.

You’ll now see a success message indicating that the subnet associations have been successfully updated.

Create and Launch an EC2 Instance

Since AWS does not allow an existing EC2 instance to be moved to another subnet, we’ll need to create a new EC2 instance in our private subnet. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



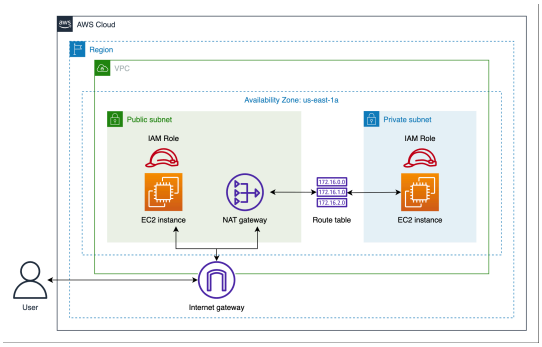
Follow the following steps to create EC2 instance in our private subnet:

- Search for “EC2” and select “EC2” from the search results to head over to the EC2 dashboard.
- Click “Instances” under the “Instances” heading in the left menu bar.
- Click the “Launch instances” button.
- Name the instance `mylab-ec2-02`.
- For “Application and OS Images (Amazon Machine Image),” select “Amazon Linux 2023 kernel-6.1 AMI.”
  - For “Architecture,” select “64-bit (x86).”
- For “Instance type,” select `t3.micro`.
- For “Key pair name - required,” under the “Key pair (login)” section, select “Proceed without a key pair (Not recommended)” from the drop-down list.
- Under the “Network settings” section, click the “Edit” button and do the following:
  - For “VPC - required,” select `mylab-vpc` from the drop-down list.
  - For “Subnet,” select `mylab-private-subnet` from the drop-down list.
  - For “Auto-assign public IP,” select “Disable” from the drop-down list, since we do not want the EC2 instance to receive any public IP address when it is launched.
  - For “Firewall (security groups),” select “Create security group.”
  - Click the “Remove” button under “Inbound security groups rules” to remove the existing rules.
- In the “Configure storage” section, make sure that “gp3,” with 8 GiB storage, is selected as the storage type.
- Click the “Advanced details” section and select `mylab-role` from the drop-down list for “IAM instance profile.”
- Leave the remaining settings as they are and click the “Launch instance” button at the end of the page.

Now that we’ve successfully launched an EC2 instance in our private subnet, let’s execute a command to verify if it has access to the internet or not.

Run a Command To Test the Internet

We’ll once again use the AWS SSM to execute a command on the EC2 instance in the private subnet. We’ll try to download the security patches and updates to the EC2 instance, thereby verifying whether or not it has access to the internet.
After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Follow the steps given below to run a command on the EC2 instance to check internet connectivity:

- Search for “Systems Manager” and select “Systems Manager” from the search results to redirect to the Systems Manager dashboard.
- Click “Run Command” under the “Node Tools” heading in the left menu bar.
- Click the “Run command” button to redirect to the “Run a command” page.
- Under the “Command document” section, search for “runshellscript” using the search bar.
  - Select “AWS-RunShellScript” from the search results.
- Under the “Command parameters” section, type the following command to download the security patches and updates to the EC2 instance in the private subnet.

```
sudo yum update -y
```

- Under the “Target selection” section, do the following:
  - For “Target selection,” select “Choose instances manually.”
  - For “Instances,” select `mylab-ec2-02` from the list.

- Under the “Output options” section, uncheck the option “Enable an S3 bucket,” since we do not want to write the command output to an S3 bucket.
- Click the “Run” button at the end of the page.

  Wait for a little while and press the button with the refresh icon at the top. Under the “Targets and outputs” section, you’ll now see “Success” in the “Status” and “Detailed Status” columns, indicating the successful execution of the command on the EC2 instance in the private subnet. This verifies that the EC2 instance could access the internet via the NAT gateway in the public subnet.

Clean Up

Now, let’s clean up the resources that we created in this lab.

Terminate the EC2 instances

We’ll start off by terminating the EC2 instances we launched.
- Head over to the EC2 dashboard by searching for “EC2” and selecting “EC2” from the search results.
- Click “Instances” under the “Instances” heading in the left menu bar.
- Select the two instances we created, `mylab-ec2-01` and `mylab-ec2-02`.
- Click the “Instance state” button and then click “Terminate (delete) instance” from the drop-down menu.
- Click the “Terminate (delete)” button in the pop-up window to confirm termination.

Delete the IAM role

Next, let’s delete the IAM role we created.

- Search for “IAM” and select “IAM” from the search results to head over to the IAM dashboard.
- Click “Roles” under the “Access management” heading in the left menu bar.
- Search for `mylab-role` using the search bar and select the role from the search results.
- Click the “Delete” button.
- Type `mylab-role` in the pop-up window and click the “Delete” button to confirm the deletion.

Delete routes

In order to delete the NAT and internet gateways, we must remove the route table entries defining routes to these gateways.

- Head over to the VPC dashboard by searching for “VPC” and selecting “VPC” from the search results.
- Click “Route tables” under the “Virtual private cloud” heading in the left menu bar to redirect to the route tables page.
- Search for the route tables associated with our subnets by typing `mylab-vpc` in the search bar.
- Perform the following steps for each of the route tables in the search results:
  - Click the ID of the route table from the search results.
  - Under the “Routes” section, click the “Edit routes” button.
  - Click the “Remove” button against the second route.
  - Click the “Save changes” button.

Delete the NAT gateway

Since we’ve deleted the routes, we can now delete the NAT gateway.

- Click “NAT gateways” under the “Virtual private cloud” heading in the left menu bar.
- On the NAT gateways page, select `mylab-nat-gateway` from the list of NAT gateways.
- Click the “Actions” button and then click “Delete NAT gateway” from the drop-down menu.
- Type “delete” in the pop-up window and click the “Delete” button to confirm the deletion.

Detach and delete the internet gateway

We can also delete the internet gateway. However, before we do that, we must detach it from our VPC.

- Click “Internet gateways” under the “Virtual private cloud” heading in the left menu bar.
- On the internet gateways page, select `mylab-internet-gateway` from the list of internet gateways.
- Click the “Actions” button and then click “Detach from VPC” from the drop-down menu.
- Click the “Detach internet gateway” button in the pop-up window.
- Select `mylab-internet-gateway` from the list of internet gateways once again.
- Click the “Actions” button and this time click “Delete internet gateway” from the drop-down menu.
- Type `delete` in the pop-up window and click the “Delete internet gateway” button to confirm the deletion.

Delete the VPC

Finally, let’s delete our VPC.

- Click “Your VPCs” under the “Virtual private cloud” heading in the left menu bar.
- Select `mylab-vpc` from the list of VPCs.
- Click the “Actions” button and then click “Delete VPC” from the drop-down menu.
- Type “delete” in the pop-up window and click the “Delete” button to confirm the deletion.

Note: Deletion of a VPC also deletes the associated subnets.

You can sign out from the AWS Management Console by clicking the username in the top right corner and clicking the “Sign out” button from the drop-down menu.